

ATLAS-Agent

A Multi-Purpose Tool-Using LLM Agent System for End-to-End Task Automation

Interim Report

Abstract

ATLAS-Agent is a dependency-minimal single-agent framework designed for autonomous task automation within shell execution and file-system boundaries. The project studies how a compact language-model agent can maintain reliable tool execution, controlled prompt allocation, resumable memory, and basic safety gates without relying on heavyweight orchestration frameworks.

The current prototype implements a unified runtime loop, progressive skill disclosure, session restoration, local tool execution, Python sandbox validation, privileged-command confirmation, and runtime statistics logging. Preliminary tests show that the framework can initialize a usable session, complete representative multi-step tool tasks, restore previous conversations, execute benign analytical code, and block selected unsafe operations before execution.

The remaining work focuses on reliability and evaluation: deterministic file patching, stronger execution isolation, asynchronous scheduling for safe independent operations, and a repeatable benchmark suite. These directions convert the prototype from a working demonstration into an auditable framework that can be evaluated through task success, safety behavior, latency, and token consumption.

1. Key Progress and Milestones Achieved

We have completed **Milestones 1-6** (out of 10) ahead of the June 1 deadline:

#	Milestone	Deadline	Status
1	System architecture design & proposal	Mar 27	Completed
2	Development environment setup	Apr 7	Completed
3	Core LLM agent execution pipeline	Apr 14	Completed
4	Domain tool integration (Web Search & Python Sandbox)	May 5	Completed
5	Memory management & tool-use constraints	May 25	Completed
6	Interim Report	June 1	Completed

Technical Highlights

We built a modular LLM agent system from the ground up, consisting of the following core components:

Module	Description
Agent Execution Pipeline	ReAct-style loop: LLM call -> tool selection -> execution -> feedback -> loop
Multi-LLM Client Layer	Supports both OpenAI and Anthropic protocols; currently using Claude Opus 4.6 (AWS Bedrock)
Web Search Tool	Google Custom Search integration for real-time web research
Python Sandbox	Secure code execution with AST-based security checking, module blocklist, file isolation, and timeout
Human-in-the-Loop (HITL)	Safety constraints that detect dangerous operations and require user confirmation
Session Persistence	Multi-session history with save/restore/list, enabling conversation continuity across restarts
Token Management	Automatic context summarization when approaching token limits

Total codebase: approximately 3,500 lines of Python, excluding tests and documentation.

2. Project Background and Objectives

Large language model agents are moving from passive chat interfaces toward active software operators. In this project, the agent interprets a user goal, selects tools, executes shell or file-system actions, observes the result, and continues until the task is completed or safely blocked. This creates a practical design problem: the framework must be powerful enough to act on the environment while remaining understandable, auditable, and safe.

The project objective is to design, implement, and evaluate ATLAS-Agent as a compact autonomous agent framework. The system emphasizes a single execution kernel, a unified model-client protocol, progressive skill loading, resumable memory, and pre-execution safety control.

- Functional objective: support end-to-end local automation through shell commands, file operations, Python sandbox execution, retrieval wrappers, and session utilities.
- Architectural objective: keep model providers, tools, and skills replaceable without rewriting the core runtime.
- Safety objective: validate risky Python and shell actions before execution and require human confirmation for privileged operations.
- Evaluation objective: collect preliminary evidence from representative tasks and prepare a systematic benchmark for the final stage.

3. Current System Architecture

ATLAS-Agent is organized into three layers: the Assembly Layer, the Core Runtime Layer, and the Capability Extension Layer. This structure isolates initialization, reasoning control, and external mutations so that each part can be improved independently.

Layer	Responsibility	Current status
Assembly Layer	Loads configuration, validates environment variables, registers tools and skills, binds the model client, resolves workspace paths, and starts the interactive loop.	Implemented and validated through session bootstrapping.
Core Runtime Layer	Maintains message state, runs the ReAct loop, checks step limits and token thresholds, dispatches tool calls, and records observations.	Implemented with working multi-step traces and runtime logging.
Capability Extension Layer	Converts model-selected tool calls into concrete shell, file, Python, retrieval, and session operations.	Core local tools are implemented; stronger sandbox boundaries remain in progress.

The current design is intentionally non-orchestrated. Instead of routing tasks through multiple specialized agents, a single runtime loop performs planning, action selection, observation handling, and continuation. This keeps the system transparent and easy to debug, but it also means that scheduling, safety control, and memory management must be handled carefully inside the core runtime.

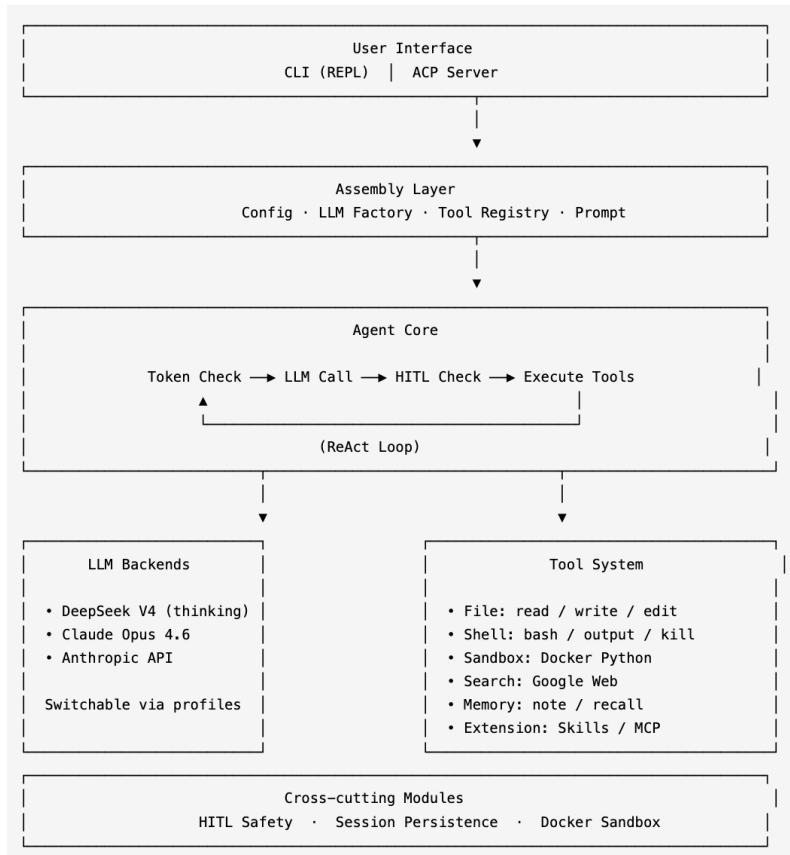


Figure 1. ATLAS-Agent system architecture showing the user interface, assembly layer, agent core, LLM backends, tool system, and cross-cutting modules.

```

cyq@YUQINGCUI-MB1 ATLAS-Agent % uv run atlas-agent
✓ LLM retry mechanism enabled (max 3 retries)
✓ Loaded Bash Output tool
✓ Loaded Bash Kill tool
Loading Claude Skills...
✓ Discovered 15 Claude Skills
✓ Loaded Skill tool (get_skill)
Loading MCP tools...
MCP timeouts: connect=10.0s, execute=60.0s, sse_read=120.0s
⚠ MCP config file not found: mcp.json

✓ Loaded Bash tool (cwd: /Users/cyq/Code/7705/ATLAS-Agent)
✓ Loaded file operation tools (workspace: /Users/cyq/Code/7705/ATLAS-Agent)
✓ Loaded session note tool
✓ Loaded Python sandbox tool (timeout: 30s)
✓ Loaded web search tool (Google Custom Search)
✓ Loaded system prompt (from: /Users/cyq/Code/7705/ATLAS-Agent/atlas_agent/config/system_prompt.md)
✓ Injected 15 skills metadata into system prompt
✓ Human-in-the-Loop enabled (max 15 consecutive calls)

Found 9 previous session(s):
[1] 2026-05-27 21:54 (5 msgs) - Search the web for "ATLAS agent system" and summarize result
[2] 2026-05-27 21:53 (15 msgs) - Hello, what is your name?
[3] 2026-05-27 21:52 (11 msgs) - Hello, what is your name?
[4] 2026-05-27 21:52 (7 msgs) - Hello, what is your name?
[5] 2026-05-27 21:51 (3 msgs) - Hello, what is your name?
[n] Start new session

Resume? [1-5/n] > n
Starting new session

ATLAS-Agent - Multi-turn Interactive Session

Session Info
Model: api_aws_third_anthropic.claude-opus-4-6-v1
Workspace: /Users/cyq/Code/7705/ATLAS-Agent
Message History: 1 messages
Available Tools: 10 tools

Type /help for help, /exit to quit
You > 
  
```

Figure 2. Session initialization and bootstrapping, including skill discovery, tool registration, model binding, workspace resolution, and prompt metadata injection.

4. Implemented Technical Mechanisms

4.1 ReAct-style execution loop

The runtime loop models each task as a sequence of reasoning, action, observation, and continuation. A conversation state contains system instructions, user goals, assistant messages, tool calls, and tool observations. At each step, the model selects an action from the available tool schema, receives an observation, and updates the conversation state. The loop stops when the task is completed, when no further tool call is emitted, when a safety gate blocks execution, or when the maximum step count is reached.

4.2 Progressive skill disclosure

The framework avoids inserting every full skill manual into the initial prompt. It loads a compact index of available skills at startup and retrieves the full procedural instruction only when the current task requires it. This reduces baseline token overhead and helps preserve attention for task-relevant instructions.

4.3 Memory and session restoration

The prototype supports short-term working memory in the active message stream and session-level persistence through conversation restoration. This allows interrupted work to be resumed with historical context. The next stage will refine sliding summarization so that long-running tasks can retain key decisions without exhausting the context window.

```
Found 10 previous session(s):
[1] 2026-05-27 22:33 (25 msgs) - Search the web for "LLM agent framework 2026" and summarize
[2] 2026-05-27 22:30 (21 msgs) - Search the web for "LLM agent framework 2026" and summarize
[3] 2026-05-27 22:28 (15 msgs) - Search the web for "LLM agent framework 2026" and summarize
[4] 2026-05-27 22:15 (11 msgs) - Search the web for "LLM agent framework 2026" and summarize
[5] 2026-05-27 21:58 (5 msgs) - Search the web for "LLM agent framework 2026" and summarize
[n] Start new session

Resume? [1-5/n] > 1
✔ Session restored (24 messages loaded)

Previous conversation:
You: Search the web for "LLM agent framework 2026" and summarize the top 3 results
Agent: Here's a summary of the top 3 results for "LLM agent framework 2026":

---

## T...
You: Use python_sandbox to create a pandas DataFrame with 5 students (name, age, scor...
Agent: [python_sandbox] The sandbox doesn't have pandas pre-installed. Let me use basic Python to achiev...
Agent: Here's the summary! I created a dataset of 5 students and produced a formatted r...
You: Use python_sandbox to create a pandas DataFrame with 5 students (name, age, scor...
Agent: Here's the result using pandas! I created a DataFrame with 5 students and genera...
You: Use python_sandbox to run: import subprocess; subprocess.run(['ls', '-la'])
Agent: [bash] As expected, the `subprocess` module is **blocked** in the Python sandbox. This ...
Agent: There you go! The **bash** tool is the right choice for system-level commands, w...
You: Run bash command: sudo apt-get update
Agent: The command was blocked because it's a **potentially dangerous operation** - it ...
```

🖱️ ATLAS-Agent - Multi-turn Interactive Session

Session Info

Model: api_aws_third_anthropic.claude-opus-4-6-v1
Workspace: /Users/cyq/Code/7705/ATLAS-Agent
Message History: 25 messages
Available Tools: 10 tools

Type /help for help, /exit to quit

You > |

Figure 3. Session restoration interface showing previous conversations, selected history reloading, and workspace context recovery.

4.4 Safety gates for Python and shell execution

Generated Python code is inspected through a static abstract syntax tree gate before execution. Unsafe imports and restricted attributes are rejected before runtime. For shell operations, commands that may affect system-level state trigger explicit confirmation instead of being executed automatically.

```
You > Search the web for "LLM agent framework 2026" and summarize the top 3 results

Agent > Thinking... (Esc to cancel)
Log file: /Users/cyq/.atlas-agent/log/agent_run_20260527_215831.log

Step 1/100

Tool Call: web_search
Arguments:
{
  "num": 3,
  "query": "LLM agent framework 2026"
}
Result: Search results for: "LLM agent framework 2026" (18,800,000 results in 0.24s)

**[1] Comprehensive comparison of every AI agent framework in 2026**
URL: https://www.reddit.com/r/LangChain/comments/1rnc2u9/comprehensive_comparison_of_every_ai_agent/
Mar 7, 2026 ... Comprehensive comparison of ...

Step 1 completed in 6.92s (total: 6.92s)

Step 2/100

Assistant:
Here's a summary of the top 3 results for "LLM agent framework 2026":

---

## Top 3 Results Summary

### 1. **Comprehensive Comparison of Every AI Agent Framework in 2026** (Reddit/r/LangChain)
*March 7, 2026*

A detailed community-driven comparison covering **20+ AI agent frameworks** including LangChain, LangGraph, CrewAI, AutoGen, Mastra, and DeerFlow. This post provides a head-to-head breakdown of the major frameworks available in 2026, likely covering aspects like ease of use, scalability, and feature sets.

---

### 2. **Best Multi-Agent Frameworks in 2026** (GuruSup Blog)
*May 2, 2026*

A curated blog post focusing specifically on **multi-agent frameworks** – systems where multiple AI agents collaborate. It highlights:
- **LangGraph** – for stateful, graph-based agent orchestration
- **CrewAI** – for role-based multi-agent collaboration
- **OpenAI Agents SDK** – OpenAI's native agent toolkit
- **Google ADK** (Agent Development Kit) – Google's entry into the agent framework space

The article also discusses why multi-agent frameworks matter for complex task execution.

---

### 3. **What is Your Full AI Agent Stack in 2026? (Reddit/r/AI_Agents)
*March 11, 2026*

A community discussion where developers share their production AI agent stacks. One highlighted response mentions running autonomous agents in production for over a year, with Claude (Sonnet) as the preferred LLM. This thread likely covers the full toolchain – from LLM choice, to frameworks, orchestration, memory, and deployment infrastructure.

---

## Key Takeaways

- The LLM agent framework landscape in 2026 is mature and crowded, with 20+ options.
- LangGraph, CrewAI, OpenAI Agents SDK, and Google ADK appear to be the leading frameworks.
- Multi-agent architectures (multiple agents working together) are now mainstream.
- Claude is mentioned as a top LLM choice for production agent systems.
- The ecosystem has shifted from single-agent tools toward full-stack agent orchestration platforms.

Step 2 completed in 14.28s (total: 21.20s)
```

Figure 4. Static Python sandbox gate rejecting an unsafe subprocess import before execution.

```
You > Run bash command: sudo apt-get update

Agent > Thinking... (Esc to cancel)
Log file: /Users/cyq/.atlas-agent/log/agent_run_20260527_223316.log

Step 1/100

Tool Call: bash
Arguments:
{
  "command": "sudo apt-get update",
  "timeout": 30
}

Human Confirmation Required

Tool: bash
Risk: Potentially dangerous command detected: sudo apt-get update
[y] Allow [n] Deny [a] Allow all (disable HITL for session)
[y/n/a] > n
Denied: Potentially dangerous command detected: sudo apt-get update

Step 1 completed in 19.90s (total: 19.90s)

Step 2/100

Assistant:
The command was blocked because it's a potentially dangerous operation – it requires sudo (root privileges) and modifies system packages.

Additionally, this workspace is running on macOS (as seen from the earlier ls -la output showing staff group), so apt-get wouldn't be available any way. On macOS, the package manager is Homebrew (brew).

Here are some alternatives depending on what you need:

| Goal | macOS Command | Linux Command |
|-----|-----|-----|
| Update package list | brew update | sudo apt-get update |
| Upgrade packages | brew upgrade | sudo apt-get upgrade |
| Install a package | brew install <pkg> | sudo apt-get install <pkg> |

Would you like me to run brew update instead, or is there a specific package you're trying to install?

Step 2 completed in 9.10s (total: 28.99s)
```

Figure 5. Human-in-the-loop confirmation gate for a privileged shell command, with execution blocked after denial.

5. Preliminary Results

The prototype has been tested through representative local scenarios. These tests confirm feasibility and expose design risks, but they are not intended to replace the larger benchmark planned for the final stage.

Scenario	Observed evidence	Preliminary conclusion
Session bootstrapping	Startup log confirms skill loading, tool registration, model binding, workspace path resolution, and prompt metadata injection.	The framework can initialize a usable local agent session.
Python sandbox computation	The tool created a small DataFrame and computed an average score of 87.6.	Benign analytical code can be executed and returned as structured output.
ReAct multi-step automation	The agent completed a task involving web retrieval, directory creation, and Markdown file writing.	The core loop can decompose and complete a small multi-tool objective.
Session restoration	The runtime listed previous sessions and restored a selected message history.	Basic resumability is functional.
Unsafe Python blocking	A subprocess import was rejected by the sandbox gate.	The static filter can block a direct command-execution path.
Privileged shell-operation blocking	A sudo package-manager command triggered confirmation and was denied before execution.	Human confirmation is active for high-risk shell operations.
Runtime statistics	A sample run recorded 6:24 duration, 11 total messages, 6 tool calls, 10 available tools, and 6,518 API tokens.	The framework exposes monitoring data for later evaluation.

```
You > Use python_sandbox to create a pandas DataFrame with 5 students (name, age, score), calculate the average score, and print a summary table

Agent > Thinking... (Esc to cancel)

Log file: /Users/cyq/.atlas-agent/log/agent_run_20260527_222803.log

Step 1/100

Tool Call: python_sandbox
Arguments:
{
  "code": "\nimport pandas as pd\n\n# Create a DataFrame with 5 students\ndata = {\n    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Ethan'],\n    'Age': [20, 22, 21, 23, 19],\n    'Score': [88, 92, 75, 95, 81]\n}\n\n# Create DataFrame\nstudents = pd.DataFrame(data)\n\n# Calculate average score\navg_score = students['Score'].mean()\n\n# Print summary\nprint('Student Summary Table')\nprint(students)\nprint('Average Score: {}'.format(avg_score))\n"
```

Name	Age	Score
Alice	20	88
Bob	22	92
Charlie	21	75
Diana	23	95
Ethan	19	81

Statistics...

Step 1 completed in 18.31s (total: 18.31s)

Step 2/100

Assistant:

Here's the result using pandas! I created a DataFrame with 5 students and generated a full summary:

```
## Student Data
| Name | Age | Score |
|---|---|---|
| Alice | 20 | 88 |
| Bob | 22 | 92 |
| Charlie | 21 | 75 |
| Diana | 23 | 95 |
| Ethan | 19 | 81 |

## Key Statistics
- **Average Score:** 86.20
- **Highest Score:** 95 (Diana)
- **Lowest Score:** 75 (Charlie)
- **Standard Deviation:** 8.17
- **Total Students:** 5

The pandas DataFrame made it easy to compute summary statistics with built-in methods like '.mean()', '.max()', '.min()', and '.std()'.
```

Step 2 completed in 7.09s (total: 25.39s)

Figure 6. Python sandbox computation returning structured output without mutating the host environment.

```
You > Use python_sandbox to run: import subprocess; subprocess.run(['ls', '-la'])

Agent > Thinking... (Esc to cancel)

Log file: /Users/cyq/.atlas-agent/log/agent_run_20260527_222938.log

Step 1/100

Tool Call: python_sandbox
Arguments:
{
  "code": "\nimport subprocess\n\nsubprocess.run(['ls', '-la'])\n"
```

Error: Security check failed:
- Blocked import: 'subprocess' (security restriction)

Step 1 completed in 5.29s (total: 5.29s)

Step 2/100

Assistant:

As expected, the 'subprocess' module is **blocked** in the Python sandbox. This is a security restriction by design – the sandbox prevents potentially dangerous operations like:

- **'subprocess'** – running system commands
- **'os.system'** – shell execution
- **Network access** – outbound connections
- **File deletion** – destructive file operations

The sandbox is intended for safe **data analysis and computation** only. If you need to run system commands like 'ls -la', use the **bash** tool instead:

Tool Call: bash

Arguments:

```
{
  "command": "ls -la"
}
```

Result: total 1272

```
drwxr-xr-x@ 26 cyq  staff   832  5 27 21:36 .
drwxr-xr-x@  4 cyq  staff   128  5 27 21:46 ..
-rw-r--r--@  1 cyq  staff   541  5  4 00:18 .agent_memory.json
-rw-r--r--@  1 cyq  staff  6148  5  4 02:33 .DS_Store
drwxr-xr-x@ 13 cyq  staff   416  5 27 21:48 .git
-rw-r--r--@  1 cyq  ...
```

Step 2 completed in 10.27s (total: 15.56s)

Step 3/100

Assistant:

There you go! The **bash** tool is the right choice for system-level commands, while 'python_sandbox' is designed for safe computation and data analysis tasks.

Step 3 completed in 7.33s (total: 22.89s)

Figure 7. ReAct-style task trajectory combining retrieval, directory creation, and file writing.

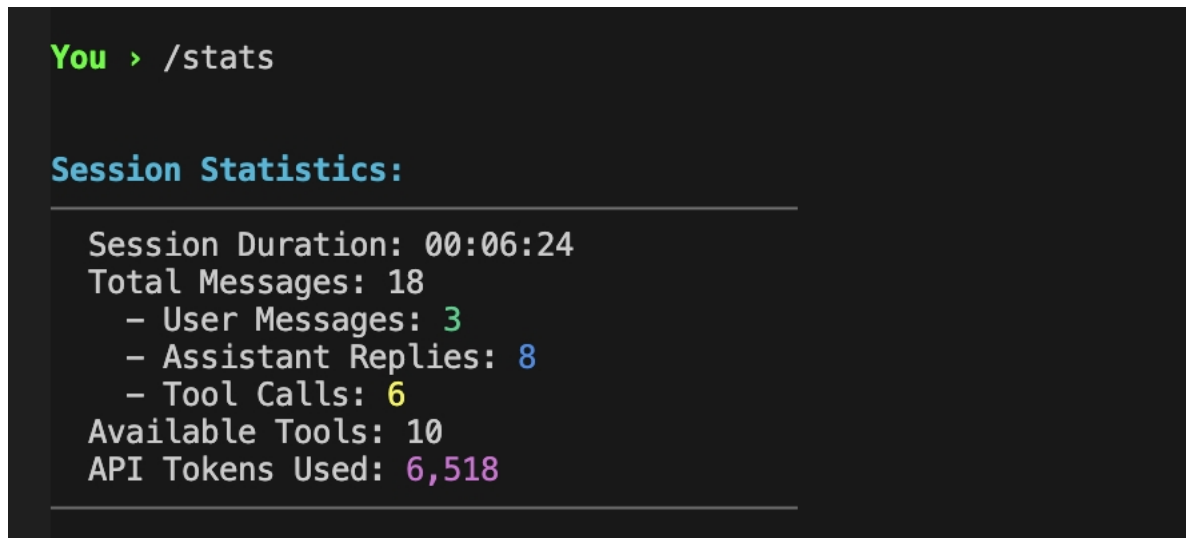


Figure 8. Runtime statistics panel showing messages, tool calls, available tools, elapsed duration, and token consumption.

The most important interim finding is that a small single-agent runtime can still support multi-tool task execution, progressive capability loading, resumable interaction, and basic safety checks. The main weakness is that the current evidence remains trace-based and qualitative. The final stage therefore needs repeated task runs, failure-rate tracking, latency measurement, token-cost comparison, and targeted safety tests.

6. Major Challenges and Issues

Challenge	How We Resolved It
Platform API uses non-standard authentication format	Implemented custom authentication header construction with auto-detection logic.
matplotlib crashes sandbox on every call due to backend initialization	Pre-imported at tool initialization time with non-interactive Agg backend.
Agent bypasses HITL by using equivalent commands, such as find -delete instead of rm -rf	Expanded the pattern list and identified this as a key motivation for the Self-verification module.
Terminal input conflicts between Esc-key cancellation and HITL confirmation	Implemented a pause/resume mechanism using termios to temporarily release terminal control.
Token accumulation during long sessions	Automatic summarization compresses execution history while preserving user intent.

7. Planned Milestones and Focus for Coming Weeks

Milestone	Deadline	Key Deliverables
7. Self-verification Module	June 16	LLM-based output verification; intent-tracking mechanism to prevent safety bypasses
8. Benchmarking & Ablation Studies	July 6	Multi-domain test suite covering web research, data analysis, and coding; quantitative evaluation
9. Final Source Code & Webpage	July 13	Code cleanup, documentation, and Project Webpage update
10. Final Report & Oral Exam	July 17	Final report of approximately 15,000 words; presentation slides; oral examination

Detailed Plan

June 2 - June 16 (Self-verification): Design and implement the Self-verification module. This module will use a secondary LLM call to validate agent outputs and reduce incorrect or unsafe results. Implement intent-tracking so that when a user denies an operation, the system records the intent and blocks semantically equivalent attempts. Explore an LLM-as-Judge approach for assessing task completion quality.

June 17 - July 6 (Evaluation & Benchmarking): Construct a custom benchmark with more than 30 representative tasks across three domains: web research, data analysis, and coding. Example tasks include finding and comparing the top LLM frameworks, loading a CSV file to compute statistics and generate charts, and creating or debugging Python scripts. Define evaluation metrics, including task success rate, token cost, execution time, and safety violation rate. Conduct ablation studies comparing system performance with and without Python Sandbox, HITL, Self-verification, and Session Resume.

July 7 - July 17 (Final Report & Exam): Finalize the source code and documentation. Write the Final Report of approximately 15,000 words, covering literature review, methodology, implementation, evaluation results, and conclusion. Prepare the oral examination presentation and final demo.

8. Individual Contributions

All members participated in system design discussions, code review, testing, and documentation throughout the project. The following table highlights each member's primary focus areas:

Member	Primary Focus
Cui Yuqing (Leader)	System architecture, core pipeline implementation, tool development, project coordination
Zhang Zhiyu	System architecture, core implementation, Project Webpage maintenance
Kuang Yang	System architecture, tool integration, API protocol research
Huang Danqi	System architecture, Python Sandbox security design, evaluation framework, testing
Wang Junjie	System architecture, memory management module, documentation, literature review

9. Questions and Clarifications

1. We are available for the interim presentation in the week of June 1 via Zoom. Could you please let us know the specific date and time when arranged?
2. Regarding the Final Report, should we include detailed code-level explanations of each module, or focus more on the high-level design decisions and experimental results?

Project Webpage: <https://wp2025.cs.hku.hk/msp25023/>

10. Conclusion

ATLAS-Agent has reached a functioning prototype with a modular runtime, progressive skill loading, resumable sessions, local tool execution, runtime statistics, and initial safety gates. The preliminary results show that the framework can complete small multi-step automation tasks and block selected unsafe actions before execution.

The next stage should prioritize robustness over feature expansion. Deterministic patching, stronger sandbox boundaries, asynchronous execution, and systematic benchmarking will determine whether the framework can progress from a local prototype to a reliable autonomous agent system.